

Mizan.ai — Development Context Handoff

Complete project context for AI-assisted development | July 15, 2026

This document is the single source of truth handed to a coding agent (Claude Code) continuing development of Mizan.ai. It consolidates all architecture decisions, deployed state, design rationale, and immediate next steps. Where this document conflicts with older files or comments in the repo, this document wins.

1. Project overview

Mizan.ai is an Arabic-first agentic AI platform for ZATCA e-invoicing compliance and VAT automation, targeting Saudi SMEs affected by ZATCA Wave 24 (threshold SAR 375,000, effective June 30, 2026; penalties from SAR 5,000 per violation). It serves a dual purpose: (1) a genuinely usable compliance product, and (2) a portfolio piece demonstrating production-grade agentic AI and Arabic-language engineering aligned with HUMAIN (PIF-owned Saudi sovereign AI company), where the developer aims to land a role. The developer (GitHub: kalimi03) works at Saudi Aramco on contract and builds this independently.

Development philosophy (follow these): iterative deployment-driven development over theoretical planning; cost-aware decisions (Modal credits, free tiers); stability over bleeding edge — mature, stable checkpoints and libraries with proven support; explicit documentation of known tradeoffs; one feature at a time — implement, test, then move on.

2. Model serving layer — DEPLOYED AND WORKING

The Modal.com serving layer (modal-serving/) is fully deployed and confirmed working end-to-end as of July 13-14, 2026, including tool-calling. Do not modify it unless explicitly asked. All inference goes through these four HTTP endpoints:

Class	Model	GPU	Endpoint	Purpose
QwenBrain	Qwen2.5-14B-Instruct-GPTQ-Int4	A10	/generate	Agent brain. Tool-calling works (Hermes-style <tool_call> parsing imple
QwenLite	Qwen2.5-7B-Instruct-GPTQ-Int4	T4	/generate-lite	Free-tier chat/RAG. No tool-calling exposed.
Translator	MADLAD-400 (3B)	T4	/translate	EN<->AR. transformers pinned <5 (v5 breaks <2ar> token).
Embedder	BGE-M3 (568M)	T4	/embed	1024-dim dense multilingual embeddings.

Key facts: all four classes authenticate to HF Hub via the 'huggingface-secret' Modal Secret (HF_TOKEN) to avoid anonymous rate-limit stalls. Weights persist on Modal Volume 'mizan-model-weights'. vLLM 0.22.1. T4 classes force dtype='float16' (no bf16 on compute capability 7.5) and TRITON_ATTN attention backend via AttentionConfig passed to LLM() — the VLLM_ATTENTION_BACKEND env var is dead in this vLLM version. VLLM_USE_FLASHINFER_SAMPLER=0 on vLLM images (no nvcc in slim images). Request/response contracts are Pydantic models in modal-serving/modal_app.py — treat them as the API spec.

History worth knowing: the brain was originally Mistral-Small-24B-AWQ; it was replaced entirely after a cascade of issues with the community AWQ repackaging (tokenizer crash, KV-cache OOM, broken tool-calling chat template). Lesson adopted: when a fragile artifact keeps generating new bug categories, replace it with a better-supported one rather than patching.

3. Feature set — finalized July 15, 2026

Feature	Status	Notes
General chat (guide bot) — Feature 1	BUILT	Always-on, no menu button. QwenLite, no tools. Scoped persona: explains Mizan, light
Ask ZATCA/VAT questions (RAG) — Feature 2	NOT BUILT	Menu button. Docling -> chunk -> BGE-M3 embed -> Qdrant -> retrieve -> QwenLite g
Chat with your document — Feature 3	PLANNED	Same RAG pipeline, different corpus: per-session/per-user Qdrant collection built from

Feature	Status	Notes
Translate a document — Feature B	PLANNED	Menu button. Wraps /translate. Gateway/agents layer handles document splitting into chunks.
Compare documents — Feature C	PLANNED	Menu button. Two related documents in, differences out. Quota: 3/month free.
ZATCA compliance calculator — Feature D	PLANNED (core differentiation)	Menu button. Multi-step HITL flow: upload docs (invoices, POs, payment details) -> Document analysis -> Compliance check -> Report generation.
Validate filled documentation	PLANNED (new)	Menu button. Takes an already-filled form, checks against the RAG knowledge base for completeness and accuracy.
Convert file format — Feature H	PLANNED	Menu button. DOCX/XLSX -> PDF only, via LibreOffice headless. Free, unlimited. PDF to DOCX/XLSX not planned.
Create new document / synthesis — Feature G	PLANNED (show 'Coming soon' in UI)	Menu button. Multi-source synthesis, max 3 inputs, 2/month free. Deferred; frontend shows a dimmed button.
Voice I/O — Feature F	CUT from v1	Whisper STT + SILMA TTS deferred entirely.
Vision model	CUT from v1	Qwen3-VL-8B dropped; no separate vision model in v1.

4. Architecture and call chain

Four layers. Browser/frontend -> app/ (FastAPI gateway: auth, tiers, quotas, sessions; the ONLY entry point the frontend talks to) -> agents/ (LangGraph orchestration + business logic per feature; calls Modal endpoints over HTTP and, later, MCP tools) -> modal-serving/ (raw inference only; no business logic ever goes here).

Frontend routing decision (important): the user explicitly selects a feature from a card grid on the home page — there is NO intent-classification layer routing free-text to features. Rationale: quotas are feature-based and easier to enforce on explicit selection; features need distinct UIs (uploads, wizards) anyway; cheaper and more reliable; the agentic complexity lives INSIDE each feature's flow (especially E), which is the more impressive engineering story.

Frontend skeleton (mocked up and approved): header with logo + nav icons (home, about, contact, reset); home view = grid of feature cards incl. the dimmed 'Coming soon' G tile; clicking a card swaps to a feature view showing only that feature's name, its options (upload area shown/hidden per feature), and a back-to-home button; a shared text input + Proceed button works in BOTH views (general chat on home, scoped to the feature when inside one); stop/run controls appear once a task is in progress (design deferred to Feature E's build). Multi-file upload design (Compare needs 2 files, Calculator needs many) deferred — single generic dropzone for now. Frontend is FastAPI-served templates + plain HTML/CSS/JS (no Flask, no React).

MCP servers: planned one per domain (VAT/ZATCA logic, translation, doc comparison, ZATCA KB/RAG) but NOT yet needed — nothing built so far requires the brain to choose between tools. MCP earns its place when Feature E's tool-choice flows are built. LangChain + LangGraph for orchestration; LangSmith for dev observability; self-hosted Langfuse planned for prod (data residency).

5. Repository structure (agreed, final)

```
Mizan-ai/
|-- docker-compose.yml          local Qdrant (+ Postgres later)
|-- app/                        FastAPI gateway (mock endpoints today; routers
|   |-- api/                    should mirror agents/ feature packages 1:1)
|   |-- web/templates/          server-rendered HTML/CSS/JS frontend
|-- modal-serving/              DEPLOYED - do not restructure
|   |-- modal_app.py
|   |-- common/model_loader.py
|   |-- test_qwen_brain.py
|-- agents/                     LangGraph orchestration layer
|   |-- requirements.txt, .env.example, README.md
|   |-- common/                 shared infra (used by 2+ features)
|   |   |-- config.py           .env loading; all endpoint URLs come from here
|   |   |-- modal_client.py     thin HTTP wrappers per Modal endpoint
|   |   |-- qdrant_client.py    (when RAG is built)
|   |   |-- extraction.py       (when needed) shared Docling wrapper - 5 features use it
|   |   |-- report_generator.py (when needed) unified PDF/DOCX/XLSX output - 3 features use it
|-- chatbot/                    Feature D [BUILT: prompts.py, state.py, graph.py, test_chatbot.py]
|-- rag/                        Feature A + A2 (A2 = same code, per-session collection)
|-- translation/                Feature B
|-- compare/                    Feature C
|-- zatca_calculator/          Feature E (HITL)
|-- validate_document/          new feature
|-- format_conversion/          Feature H
|-- synthesis/                  Feature G (v2)
|-- mcp_servers/                when tool-choice becomes needed
```

Conventions established by the built code: pip + venv; config via .env loaded in common/config.py (never read os.environ elsewhere); thin no-retry HTTP clients in common/modal_client.py; each feature is a package with prompts.py / state.py (TypedDict) / graph.py (LangGraph StateGraph, even when single-node, plus a plain-function entry point) / test.py (manual smoke tests, run-and-eyeball with hard asserts only where behavior is unambiguous — mirroring modal-serving/test_qwen_brain.py). System prompts are prepended exactly once and never duplicated across turns.

6. Known open items and cleanup debt

- app/modal_app.py is a broken stale duplicate of modal-serving/modal_app.py (imports a common/ package that doesn't exist next to it). Delete it or fix it; never deploy from it.
- app/app.py and app/files.zip are unrelated leftovers from a different project ('Product Similarity API'). Remove when convenient.
- Translator and Embedder endpoints were confirmed via deploy logs only — never exercised with real end-to-end requests. Test them when translation/RAG features first call them.
- No automated test suite anywhere — only manual smoke-test scripts by convention.
- No real ZATCA/VAT reference documents ingested yet — RAG tests use obviously-fake placeholder content until real docs are provided.

7. Infrastructure plan

Local dev NOW: laptop + Docker Desktop (Docker + Kubernetes enabled, kubectl working). Qdrant runs locally via docker compose. Kaggle notebooks for any GPU work (quantization).

Integration NEXT: once all features work individually, deploy the whole stack on the local Docker Desktop Kubernetes to validate end-to-end.

Production LATER: GCP (decision made July 14, 2026 — supersedes ALL earlier AWS plans: no EC2/k3s, no AWS Secrets Manager, no S3/EBS). GCP equivalents: GCE VM or GKE, Cloud Storage, Secret Manager, persistent disks. Exact GCP architecture TBD. Model inference stays on Modal regardless.

Known documented tradeoff: Modal is US-hosted — a deliberate PDPL/data-sovereignty gap for the prototype phase; PDPL-compliant in-Kingdom hosting is flagged as a production requirement.

8. Hard-won lessons (do not re-learn these)

- T4 = compute capability 7.5: no bf16 (always dtype='float16' explicitly), no FlashAttention2, use TRITON_ATTN via AttentionConfig object, not env var.
- transformers must stay <5 in the translate image: v5 splits MADLAD's <2ar> token into two IDs -> degenerate output. Load-time sanity check exists in model_loader.py.
- Python logging defaults drop logger.info() silently (no basicConfig anywhere); use print() for diagnostics on Modal — stdout is always captured. 'No output' is not 'nothing happened'.
- When docs/source-code reasoning contradicts observed behavior twice, stop reasoning and read ground truth (actual files, logs, output).
- A fix for one failure can silently create another (tokenizer_mode='hf' fixed a crash but broke tool-calling). Re-test surrounding functionality after any workaround.
- Prefer removing a failure's trigger over suppressing its symptom.
- Warnings that appear in every log deserve action, not dismissal (anonymous HF Hub warning eventually caused a 15-minute download stall; fixed with HF_TOKEN secret).

9. Immediate next steps (in order)

1. Run and validate the chatbot smoke tests against the live QwenLite endpoint (python -m chatbot.test_chatbot). Scrutinize the redirect tests hardest — if the 7B model leaks ZATCA answers, iterate on chatbot/prompts.py only.
2. Build rag/ (Feature A) per section 3's design: chunking, ingest CLI (Docling -> embed -> Qdrant upsert), retrieve (embed query -> threshold-filtered search), two-node LangGraph (retrieve -> generate on QwenLite), smoke tests with clearly-fake placeholder content.
3. Promote Docling extraction into common/extraction.py the moment a second feature needs it.
4. Wire app/ (FastAPI gateway) to the built agents features; replace mock endpoints; build the approved frontend skeleton (home grid <-> feature view state machine).
5. Continue features in order: translation -> compare -> zatca_calculator (introduces QwenBrain tool-calling + MCP + HITL) -> validate_document -> format_conversion.
6. Local Kubernetes integration test, then GCP.

10. Standing constraints for the coding agent

- Never put business logic in modal-serving/ — it is inference-only by contract.
- QwenLite for chat and RAG generation; QwenBrain (tool-calling) reserved for Feature E and future tool-choice flows — this tier split is a deliberate cost decision.
- The guide chatbot must never answer ZATCA/VAT questions directly — redirect-only, enforced by prompt.
- RAG must answer only from retrieved content and admit when the KB lacks an answer — wrong compliance answers cost users real money.
- Keep dependencies minimal and stable; add per-feature deps only when that feature is built.
- Follow the established package conventions (section 5) for every new feature.
- Arabic + English must both work in every user-facing text path.
- Currency constants when needed: 1 USD = 95 INR, 1 SAR = 25.1 INR (fixed until updated).